Лабораторна робота №6

«Робота з різними типами даних та моделями та відображення списками»

Мета

Сформувати практичні навички роботи з базовими структурами даних у мові Dart (List, Map), навчитися створювати та використовувати власні моделі даних (model class), застосовувати enum для опису станів і типів об’єктів, а також реалізовувати відображення списків та оновлення інтерфейсу у Flutter-застосунку з використанням механізму зміни стану.

Теоретичні відомості

1. List(список)

List — впорядкована колекція елементів одного типу.

Приклади використання: список завдань, рецептів, фільмів, покупок.

Поширені операції:

- add(), remove(), removeAt()
- length
- where(...) (фільтрація)
- map(...) (перетворення)

```dart
final List<String> items = ['Пункт 1', 'Пункт 2'];

items.add('Пункт 3');
items.removeAt(0);

final int count = items.length;
```

2. Map (словник «ключ → значення»)

Map зручно зберігає дані у вигляді пар: ключ і значення. Приклади: статистика, налаштування, підрахунок категорій

```
final Map<String, int> stats = {
 'completed': 0,
 'favorites': 0,
};

stats['completed'] = (stats['completed'] ?? 0) + 1;
```

3. enum (перелік фіксованих значень)

enum використовується, коли є обмежений набір станів/типів. Приклади: пріоритет, категорія, статус, складність.

```
enum Priority { low, medium, high }

Priority priority = Priority.medium;
```

4. Model (клас-модель даних)

Модель описує один “об’єкт” предметної області (Завдання, Рецепт, Контакт тощо).

Навіщо model:

- зручно зберігати/передавати дані;
- легко розширювати (додати поле, метод);
- простіше будувати UI зі списків.

```
enum ItemStatus { active, done }

class ItemModel {
 final int id;
 String title;
 bool isFavorite;
 ItemStatus status;
```

```
       ItemModel({
         required this.id,
         required this.title,
         this.isFavorite = false,
         this.status = ItemStatus.active,
       });
      }
```

5. Віджет ListView

ListView — це віджет для відображення прокручуваного списку елементів.

Існує кілька способів створення списку:

- ListView() — коли елементів небагато;
- ListView.builder() — коли список динамічний або великий.

```
ListView.builder(
 itemCount: items.length,
 itemBuilder: (context, index) {
   return ListTile(
     title: Text(items[index]),
   );
 },
);
```

Пояснення:

- itemCount — кількість елементів списку;
- itemBuilder — функція, що будує кожен елемент;
- index — позиція елемента в списку.
- items[index] — доступ до конкретного елемента списку.

6. Віджет ListTile

ListTile — це спеціалізований віджет для створення одного рядка списку. Найчастіше використовується разом із ListView.

```
ListTile(
 leading: Icon(Icons.star),
 title: Text('Назва елемента'),
 subtitle: Text('Додаткова інформація'),
 trailing: Icon(Icons.delete),
 onTap: () {
   // дія при натисканні
 },
)
```

Опис властивостей:

- leading — віджет зліва (зазвичай іконка або аватар);
- title — основний текст елемента;
- subtitle — додатковий текст під заголовком;
- trailing — віджет справа (кнопка, іконка, перемикач);
- onTap — обробник натискання на весь елемент;
- onLongPress — обробник довгого натискання.

Зміна стану елемента в ListTile

Оскільки ListTile часто представляє один об’єкт списку, у ньому можна змінювати стан:

```
ListTile(
 title: Text(items[index].title),
 trailing: IconButton(
   icon: Icon(
     items[index].isFavorite
         ? Icons.favorite
         : Icons.favorite_border,
   ),
   onPressed: () {
```

```
      setState(() {
        items[index].isFavorite =
        !items[index].isFavorite;
      });
    },
  ),
);
```

У цьому прикладі:

- змінюється поле об’єкта списку;
- викликається setState();
- інтерфейс автоматично оновлюється.

Коли використовують ListTile:

- у списках завдань;
- у налаштуваннях;
- у списках контактів;
- у каталогах товарів або фільмів.

```
Expanded(
  child: ListView.builder(
    itemCount: items.length,
    itemBuilder: (context, index) {
      final item = items[index];

      return ListTile(
        leading: Icon(
          item.isFavorite ? Icons.favorite : Icons.favorite_border,
          color: item.isFavorite ? Colors.red : null,
        ),
        title: Text(
          item.title,
          style: TextStyle(
            decoration:
                                        item.status   ==   ItemStatus.done   ?
TextDecoration.lineThrough : null,
```

```
          color: item.status == ItemStatus.done
              ? (isDarkMode ? Colors.white70 : Colors.black54)
              : null,
        ),
      ),
          subtitle: Text(item.status == ItemStatus.done ? 'Виконано' :
'Активне'),
      trailing: IconButton(
        icon: const Icon(Icons.delete),
        onPressed: () {
          setState(() {
            items.removeAt(index);
          });
        },
      ),
      onTap: () {
        setState(() {
          item.isFavorite = !item.isFavorite;
        });
      },
      onLongPress: () {
        setState(() {
          item.status = item.status == ItemStatus.done
              ? ItemStatus.active
              : ItemStatus.done;
        });
      },
    );
  },
),
),
```

Завдання на лабораторну роботу

1. Продовжити роботу над проєктом, розпочатим у Лабораторній роботі №5.
2. Створити окремий файл моделі (наприклад, item_model.dart).
3. Описати class Model з мінімум 4 полями: int, string, bool, enum
4. Створити List<Model> — основне джерело даних екрану (мінімум 5 початкових елементів).
5. Відобразити список з ListView.builder та ListTile).
6. Реалізувати додавання, видалення елемента.
7. Реалізувати зміну стану елемента, наприклад:
    - іконка ⭐ “обране” перемикає isFavorite;
    - або зміна значення status (enum) кнопкою/довгим натисканням;
    - оновлення через setState().
8. Створити Map<String, int> для статистики, наприклад:
    - загальна кількість елементів;
    - кількість обраних;
    - кількість виконаних (status = done).

Контрольні запитання

1. Чим List відрізняється від Map у Dart?
2. Для чого використовують enum? Наведіть приклад.
3. Що таке модель даних (model class) і яку роль вона відіграє у Flutter-застосунку?
4. Як додати елемент у List<Model> та оновити інтерфейс?

5. Яка різниця між методами map() і where() для списку?
6. Як реалізувати фільтрацію списку (наприклад, показувати лише обрані елементи)?
7. Для чого використовується віджет ListView.builder? У чому його перевага?
8. Що таке itemCount, itemBuilder та index у ListView.builder?
9. Як змінити стан окремого елемента списку та оновити UI?
10. Що станеться, якщо змінити дані моделі без виклику setState()?